

Entrega 0: Plan de proyecto

Programación Distribuida y Componentes · Proyecto Final

El objetivo de esta entrega es definir la idea general del proyecto considerando los módulos que luego deberán ser cumplidos y certificados.

Nota de esta versión: además del plan, esta entrega ya incluye una parte muy grande del sistema implementado y funcionando (ver “Estado al momento de esta entrega”). Los campos marcados <completar> son datos personales/administrativos que no puedo completar por mi cuenta.

Datos generales

Campo	Valor
Grupo N°	<completar>
Tag / Release de esta entrega	v0-plan-del-proyecto
Fecha de entrega	08/09/2026

Proyecto

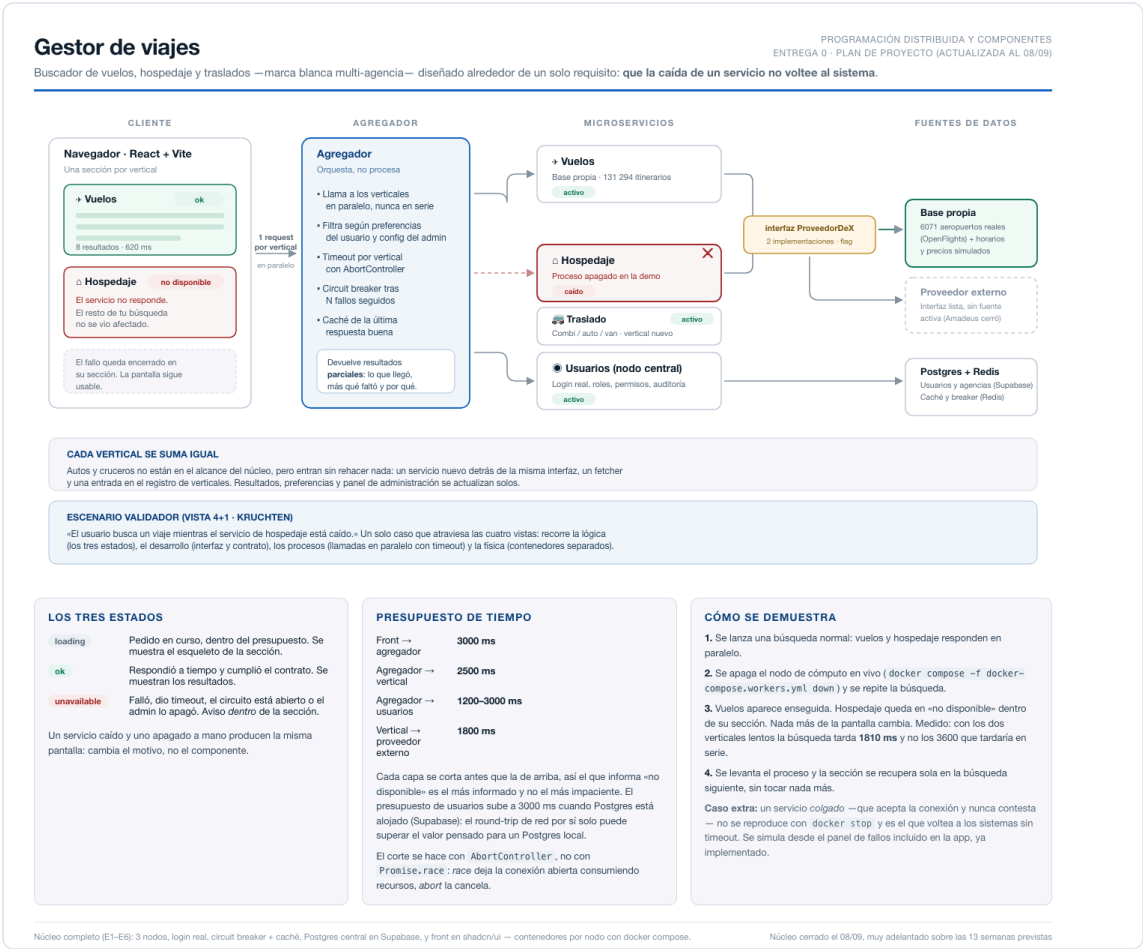
Área: Turismo — comercio electrónico de servicios de viaje, con arquitectura de marca blanca multi-agencia.

Empresa o entidad asociada: <ninguna: proyecto académico / completar si corresponde>

Título: Gestor de viajes — buscador multivertical con tolerancia a fallos parciales y arquitectura distribuida en tres nodos.

Dominio: Búsqueda y consulta de disponibilidad de servicios de viaje (vuelos, hospedaje y traslados), ofrecido como plataforma de marca blanca a varias agencias desde un mismo sistema, con datos provistos por una interfaz intercambiable entre una API externa de la industria y una base propia de respaldo.

Breve descripción: Un buscador de viajes estilo Booking.com — vuelos, hospedaje y traslados — que consulta los tres verticales en paralelo y muestra los resultados diferenciados según qué buscó el usuario. Lo que distingue al proyecto no es el buscador sino su comportamiento ante fallos: microservicios independientes, repartidos en tres nodos físicos (central, de cómputo y uno por agencia contratante), coordinados por un agregador con timeouts, circuit breaker y caché de última respuesta buena, de modo que la caída de cualquier servicio degrada únicamente su propia sección de la pantalla. Tiene login real con roles y permisos, y cada vertical obtiene sus datos a través de una interfaz con dos implementaciones intercambiables —la API real de Amadeus y una base propia con datos reales de referencia (aeropuertos, aerolíneas y rutas del mundo entero)— seleccionables por configuración.



Descripción

El problema

Un buscador de viajes depende de varias fuentes de datos que no controla: proveedores de vuelos, de hospedaje, de traslados. En la mayoría de las implementaciones ingenuas, la pantalla espera a que **todas** respondan antes de mostrar algo. Alcanza con que una sola se caiga —o peor, que quede colgada sin responder— para que el usuario vea una pantalla en blanco, un error genérico o una espera indefinida, aunque el 80 % de la información que pidió ya esté disponible. El proyecto toma ese problema como requisito central de diseño, no como un detalle de manejo de errores a resolver al final.

Qué hace el sistema

El usuario puede buscar sin cuenta: elige una solapa —Vuelos, Hospedaje o Traslado—, cada una le pide sólo los campos que a ese servicio le importan, y el sistema devuelve la sección correspondiente a lo que pidió; el resto queda colapsado con el motivo, no oculto sin rastro. Con sesión iniciada —ofrecida, nunca forzada— puede indicar en su perfil qué verticales le interesan, y un administrador puede habilitar o deshabilitar verticales para toda su agencia y gestionar permisos de otros administradores.

El sistema es además **multi-agencia (marca blanca)**: el mismo backend sirve a varias agencias contratantes (`ag-demo` , `ag-sur` en la semilla), cada una con su propio branding, su propia configuración de verticales y su propio nodo de agencia, todas leyendo y escribiendo del mismo nodo central.

Arquitectura: tres nodos físicos

Nodo central — entrada y almacenamiento compartido entre todas las agencias:

- **Agregador** (API Gateway) — recibe la búsqueda, decide a qué verticales llamar según la solapa elegida, las preferencias del usuario y la configuración del administrador, los llama **en paralelo**, arma la respuesta con lo que haya llegado a tiempo, y es quien emite y verifica la sesión (JWT).
- **Usuarios** — autenticación, roles, permisos, preferencias y auditoría. Única base **realmente centralizada** del sistema, en Postgres alojado en Supabase.
- **Redis** — caché de última respuesta buena y estado del circuit breaker. Deliberadamente *no* centralizado junto con Postgres: es estado por nodo.

Nodo de cómputo — un microservicio por vertical, cada uno con su propia base SQLite, sin acceso a la de los otros: **Vuelos** (6071 aeropuertos, 131 294 itinerarios), **Hospedaje** (48 568 alojamientos), **Traslado** (combi, auto privado, van o taxi; 6071 ciudades con tarifa).

Nodo de agencia — una instancia por agencia contratante: su propio front de marca blanca más un servicio `nodo-agencia` que guarda una bitácora local de cada búsqueda y la sincroniza al nodo central.

El cliente es una SPA en React (shadcn/ui) que renderiza una sección por vertical; cada sección hace su propio pedido al agregador, de manera que su ciclo de vida es independiente del de las demás.

Cómo se consigue la tolerancia a fallos

Cinco mecanismos, todos implementados y verificados:

1. **Llamadas en paralelo.** El tiempo total de una búsqueda es el del vertical más lento, no la suma de todos (medido: 2065 ms con dos verticales lentos en paralelo).
2. **Timeout por tramo con `AbortController`.** 3000 ms front→agregador, 2500 ms agregador→vertical, 1200–3000 ms agregador→usuarios (sube si Postgres está alojado, como hoy), 1800 ms vertical→Amadeus. El corte se hace con `AbortController`, no `Promise.race`: `race` deja la conexión abierta consumiendo un socket, `abort` la cancela de verdad.
3. **Modelo de tres estados.** `loading` / `ok` / `unavailable`, el mismo enum en front y back. La sección `unavailable` muestra el motivo dentro de su propia caja.
4. **Circuit breaker.** Si un vertical falla 3 veces seguidas, el agregador deja de esperarlo 30 s y responde sin él. Verificado con prueba automatizada.
5. **Caché de la última respuesta buena.** Si el circuito está abierto, se muestran resultados de los últimos 10 minutos en vez de nada. Corregido durante el desarrollo un bug real: la caché sólo debe entrar en juego con el circuito abierto, no en cualquier fallo aislado — si no, esconde la clase de falla que el proyecto tiene que demostrar.

Búsqueda diferenciada y catálogo global

El buscador tiene solapas (Vuelos / Hospedaje / Traslado); cada una pide sólo los campos que le hacen falta y la pantalla de resultados muestra únicamente la sección pedida. El catálogo de lugares es el dataset **completo** de OpenFlights (licencia ODbL): 6071 aeropuertos, 983 aerolíneas activas y 65 647 rutas reales, de todo el mundo.

Login real y permisos

Se puede navegar y buscar sin cuenta; perfil y administración exigen sesión. Contraseñas con `scrypt` (`node:crypto`), sesión con JWT armado a mano (HS256, sin librería de auth), verificación de email simulada (código en la respuesta, etiquetado “modo demo”). Administradores con permisos granulares (`gestion_verticales`, `gestion_usuarios`); cada cambio de configuración queda en una tabla de auditoría consultable por API.

Fuente de datos

Cada vertical con proveedor externo (vuelos, hospedaje) define una interfaz propia con dos implementaciones intercambiables por variable de entorno: base propia o la API real de Amadeus. El plan original usaba el tier Self-Service gratuito de Amadeus; ese portal fue dado de baja el **17 de julio de 2026**, a mitad de camino del proyecto. El sistema no se vio afectado — nunca dependió de Amadeus, sino de una interfaz. La implementación contra Amadeus sigue completa y sin usar en el repositorio, como prueba en vivo de que la abstracción cumple su función.

Cómo se demuestra

1. Se lanza una búsqueda normal; los verticales responden en paralelo.
2. Se apaga el nodo de cómputo entero delante del evaluador (`docker compose -f docker-compose.workers.yml down`) y se repite la búsqueda.
3. La sección afectada pasa a “no disponible” **dentro de su sección**. Ningún otro elemento de la pantalla cambia.
4. Se levanta de nuevo y la sección se recupera sola en la búsqueda siguiente.

Un caso adicional, el servicio *colgado* (acepta la conexión y nunca contesta), se simula desde el panel de fallos incluido en la aplicación.

Alcance

Dentro: vuelos, hospedaje, traslado, login real con roles y permisos, agregador con circuit breaker y caché, preferencias en dos capas, panel de administración con auditoría, arquitectura de tres nodos con marca blanca multi-agencia, y despliegue en contenedores.

Fuera del núcleo: autos y cruceros — no por dificultad sino por tiempo; el vertical está armado como un patrón, así que sumarlos es agregar código nuevo, no rehacer el existente.

Condicionado al tiempo disponible: reservas con concurrencia real (Etapa 7) — la única etapa del plan original que sigue pendiente.

Mapeo a los módulos de la materia

Módulo	Cómo lo cubre este proyecto
1 · Vistas 4+1	Documentado en las cuatro vistas, validadas con un escenario único: “ <i>el usuario busca un viaje mientras el servicio de hospedaje está caído</i> ”, que atraviesa lógica, desarrollo, procesos y física (tres nodos en contenedores separados).

Módulo	Cómo lo cubre este proyecto
2 · Infraestructura	Microservicios con base propia por servicio, tres nodos físicos simulados con <code>docker compose</code> ; Postgres central migrado a Supabase; caché como transparencia de replicación con su costo visible.
3 · Comunicación y sincronización	Llamadas en paralelo con timeout y cancelación; bitácora de cada nodo de agencia sincronizada al central.
4 · Componentes y contratos	Interfaces <code>ProveedorDeVuelos</code> / <code>ProveedorDeHospedaje</code> con dos implementaciones por variable de entorno. Contrato JSON escrito antes que el código.
5 · Seguridad	Login real (<code>scrypt</code> + JWT propios), roles y permisos validados en el servidor, secretos fuera del repositorio, auditoría de configuración.
6 · Patrones y escalabilidad	Circuit breaker y caché de última respuesta buena, verificados con pruebas automatizadas.

Estado al momento de esta entrega

El núcleo completo (Etapas 1 a 6) está implementado y el sistema es ejecutable de punta a punta, bastante antes de lo que preveía el plan original.

Seis servicios independientes en tres nodos (agregador, usuarios, vuelos, hospedaje, traslado, nodo-agencia); autenticación real completa; dos agencias de marca blanca sembradas y aisladas entre sí; frontend en React sobre shadcn/ui con búsqueda diferenciada por solapa; contrato escrito antes que el código; despliegue en contenedores por nodo (`docker-compose.central.yml` , `.workers.yml` , `.agencias.yml`); pruebas automatizadas en verde contra el sistema real, incluido el circuit breaker completo, el flujo de login de punta a punta y la medición de que las llamadas en paralelo ahorran tiempo real.

Lo único que queda del plan original sin encarar es la **Etapas 7, reservas con concurrencia real** — opcional, condicionada al tiempo que quede tras cerrar la documentación y el ensayo de la defensa.

Planificación



Planificación (Gantt)

Etapas	Módulos	Previsto	Estado
E0 · Plan del proyecto	—	01/09 – 07/09	✓ Hecho
E1 · Cimientos y contrato	1, 4	01/09 – 06/09	✓ Hecho
E2 · Backend distribuido	2, 3	01/09 – 06/09	✓ Hecho
E3 · Identidad y preferencias	3, 5	07/09 – 27/09	✓ Hecho
E4 · Proveedor externo	4, 5	28/09 – 11/10	✓ Hecho
E5 · Resiliencia y despliegue	2, 6	12/10 – 15/11	✓ Hecho
E6 · Administración	6	16/11 – 29/11	✓ Hecho
Núcleo			✓ Completo al 08/09
E7 · Reservas (opcional)	3, 6	30/11 – 20/12	⌚ Pendiente

Replanificación real: el núcleo (E1–E6) cerró en la primera semana en vez de las trece previstas. El tiempo liberado se reinvertió en profundizar más allá del alcance original de esta versión: arquitectura de tres nodos con marca blanca multi-agencia, catálogo global de aeropuertos (6071, antes 70), un vertical nuevo (Traslado, en vez de Seguros), búsqueda diferenciada por solapa, migración del Postgres central a Supabase, y rediseño de interfaz sobre shadow/ui.

Actividades

Etapa / Épica	Sprint	Estado	Tareas principales
E0 · Plan del proyecto	S0 · Arranque	✓	Definición de alcance, arquitectura e infografía. Repositorio y plan.
E1 · Cimientos y contrato	S1 · Cascarón	✓	Contrato JSON del agregador. Front con las pantallas y el modelo de tres estados. Panel de simulación de fallos.
E2 · Backend distribuido	S2 · Microservicios	✓	Servicios de vuelos, hospedaje y usuarios con base y semilla propias. Interfaz de proveedor. Agregador en paralelo con timeouts.
E3 · Identidad y preferencias	S3 · Sesión y perfil	✓	Registro y login real con JWT y <code>scrypt</code> . Verificación de email simulada. Roles y permisos validados en el servidor. Auditoría.
E4 · Proveedor externo	S4 · Proveedor	✓	Implementación contra Amadeus tras la interfaz. Evaluación de alternativas tras su cierre. Catálogo global de referencia.
E5 · Resiliencia y despliegue	S5 · Breaker y caché	✓	Circuit breaker y caché gateada al circuito abierto.
	S6 · Contenedores y multi-nodo	✓	<code>docker compose</code> por nodo. Migración a Supabase. Nodo de agencia. Vertical

Etapa / Épica	Sprint	Estado	Tareas principales
			Traslado. Búsqueda diferenciada.
E6 · Administración	S7 · Panel de admin	✅	Tablero de preferencias, gestión de administradores y permisos, auditoría a la vista. Rediseño sobre shadcn/ui.
E7 · Reservas <i>(opcional)</i>	S8 · Reservas	🕒	Reserva con control de concurrencia e idempotencia.

Participación

Integrantes

	Nombre y apellido	Legajo	Correo
A	⟨completar⟩	⟨completar⟩	⟨completar⟩
B	⟨completar⟩	⟨completar⟩	⟨completar⟩

(No hay forma de saber desde el sistema si el proyecto es individual o en equipo, ni quiénes lo integran — completar esta tabla, o borrar la fila B si es individual, antes de entregar.)

Reparto del trabajo

El reparto sigue el corte natural del sistema: un camino va del navegador al agregador, el otro del agregador a las fuentes de datos y la infraestructura. El contrato JSON del agregador —definido antes de escribir código— permite avanzar en paralelo sin bloquearse. Si el proyecto termina siendo individual, esta misma división describe el orden en que se fue encarando el trabajo.

Cliente y orquestación: aplicación React sobre shadcn/ui (buscador por solapas, resultados diferenciados, perfil, panel de administración); modelo de tres estados; agregador (llamadas en paralelo, timeouts, sesión JWT, circuit breaker, caché); panel de simulación de fallos.

Servicios, datos e infraestructura: microservicios de vuelos, hospedaje y traslado con sus bases y el catálogo global de referencia; interfaz `ProveedorDeX` e integración con Amadeus; servicio de usuarios (login real, roles, permisos, auditoría); arquitectura de tres nodos y `docker compose`; migración del Postgres central a Supabase; gestión de secretos.

Compartido: contrato del agregador y decisiones de arquitectura; documentación en las cuatro vistas de Kruchten; revisión cruzada de código; ensayo y presentación de la demostración final.

Espacio para el docente

Nota: ____ / 10

Corregido por: <completar>

Fecha de corrección: <completar>

Observaciones: